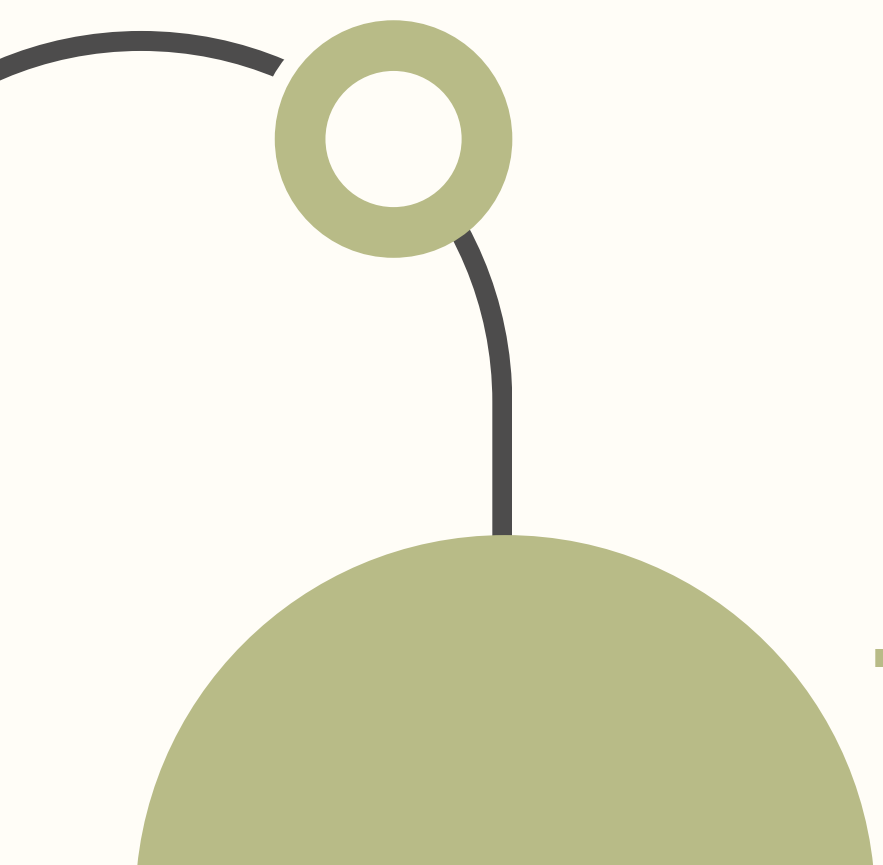


→

Presentación

COMPILADOR AKIRANI

TRABAJO HECHO POR:
GONZALEZ GASTELUM MARCO VINICIO



GRAMATICA DE AKIRANI

Program ::= class **Identifier** [**VarDeclaration**]* { **Statement** } <EOF>

VarDeclaration ::= **Type Identifier** ;

Type ::= boolean | int

Statement ::= { [**Statement**]* }

 | while (**Expression**) **Statement**

 | println (**Expression**) ;

 | **Identifier** = **Expression** ;

Expression ::= **Identifier** (< | + | - | *) **Identifier**

 | true | false | **Identifier** | **Integer**

Identifier ::= **letter** [**letter** | **digit**]*

Integer ::= **digit** ⁺

Letter ::= a | b | c | ... | z | A | B | C | ... | Z

Digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Gramatica del lenguaje BNF (backus naur form)

G = {SI, P, NT, T}

SI = {PROGRAM}

NT = {PROGRAM, VARDECLARATION,
TYPE, STATEMENT, EXPRESSION,
IDENTIFIER, INTEGER, LETTER, DIGIT}

T = { class, { , }, ; , <EOF>, boolean, int,
while, (,) , println , = , < , + , - , * , true,
false, a, b, c,...z, A, B, C,...Z, 0, 1,...9}

Programa de ejemplo

```
class prueba
```

```
int x;
```

```
int y;
```

```
int z;
```

```
{
```

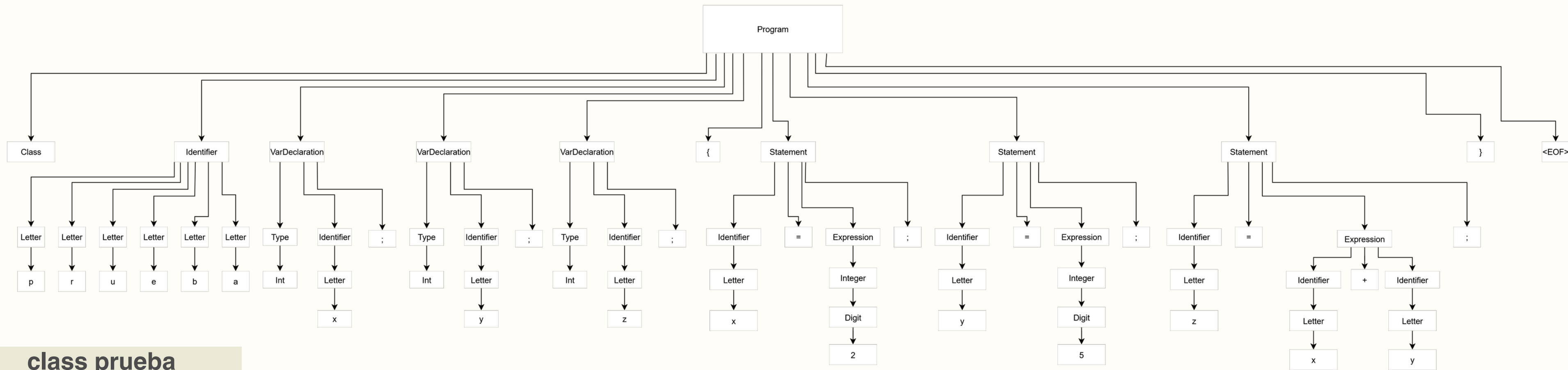
```
  x = 2;
```

```
  y = 5;
```

```
  z = x + y;
```

```
}
```

Árbol de parsing



class prueba

int x;

int y;

int z;

{

x = 2;

y = 5;

z = x + y;

}

```
class prueba
```

```
int x;
```

```
int y;
```

```
int z;
```

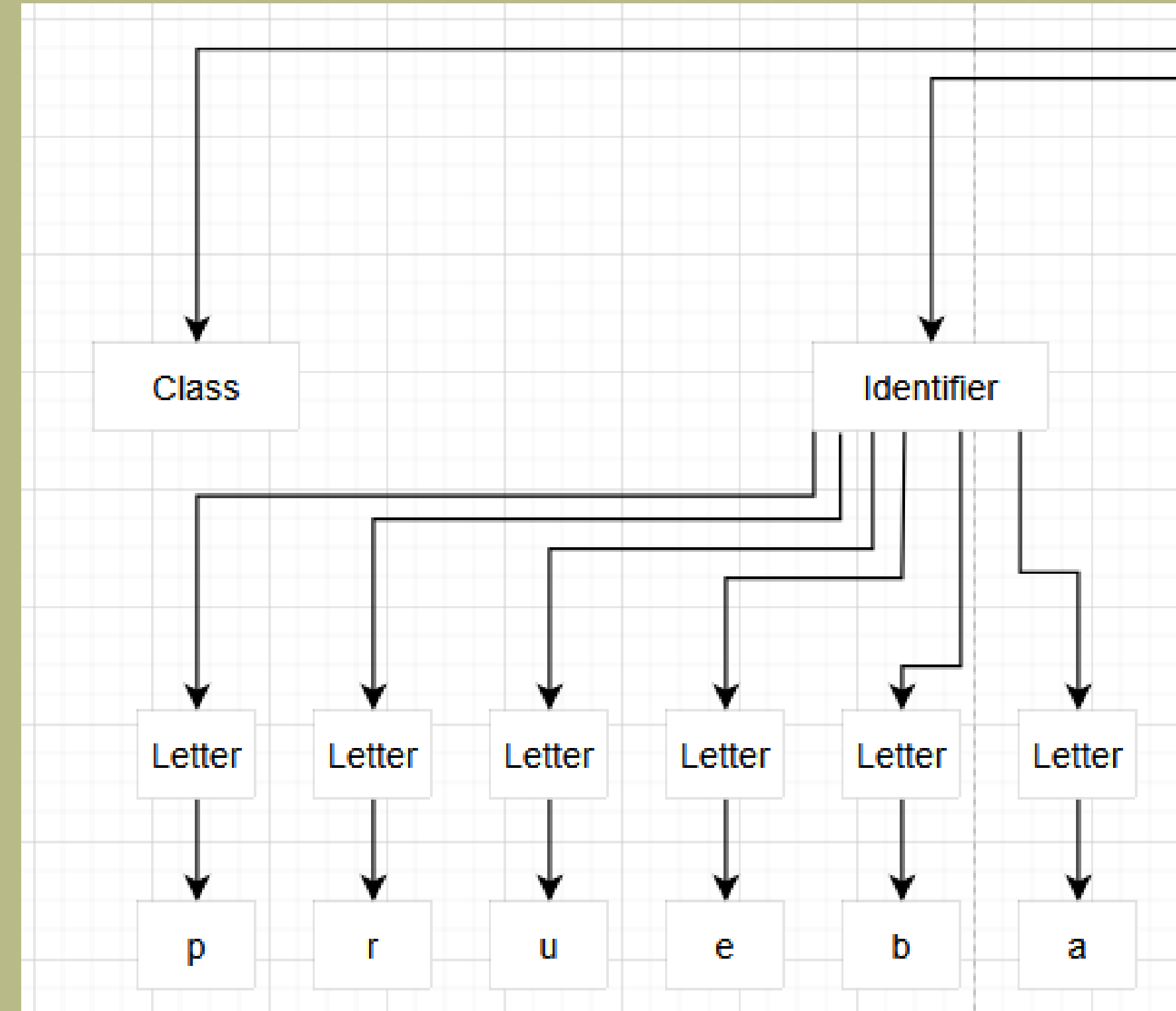
```
{
```

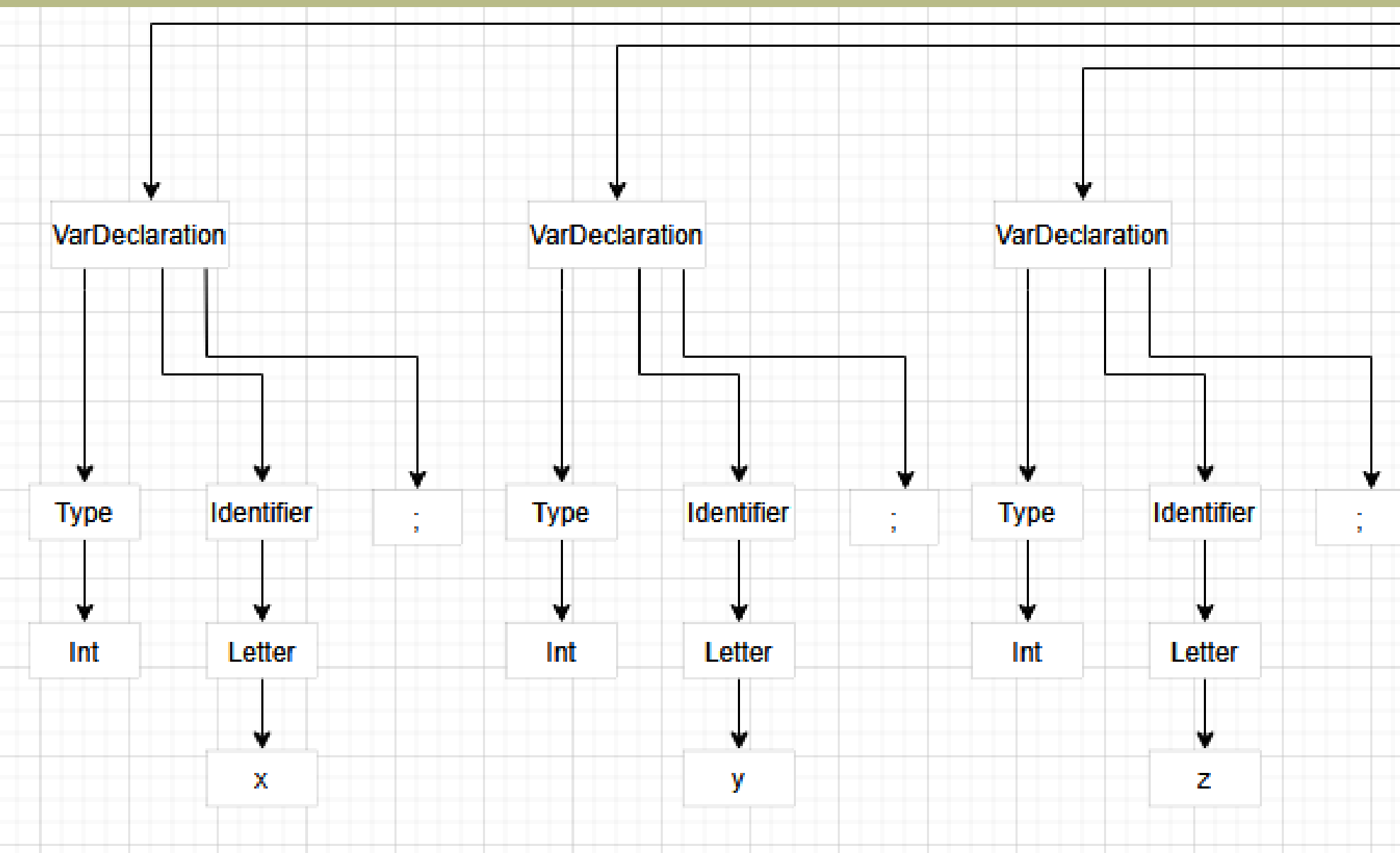
```
  x = 2;
```

```
  y = 5;
```

```
  z = x + y;
```

```
}
```





class prueba

int x;

int y;

int z;

{

x = 2;

y = 5;

z = x + y;

}

```
class prueba
```

```
int x;
```

```
int y;
```

```
int z;
```

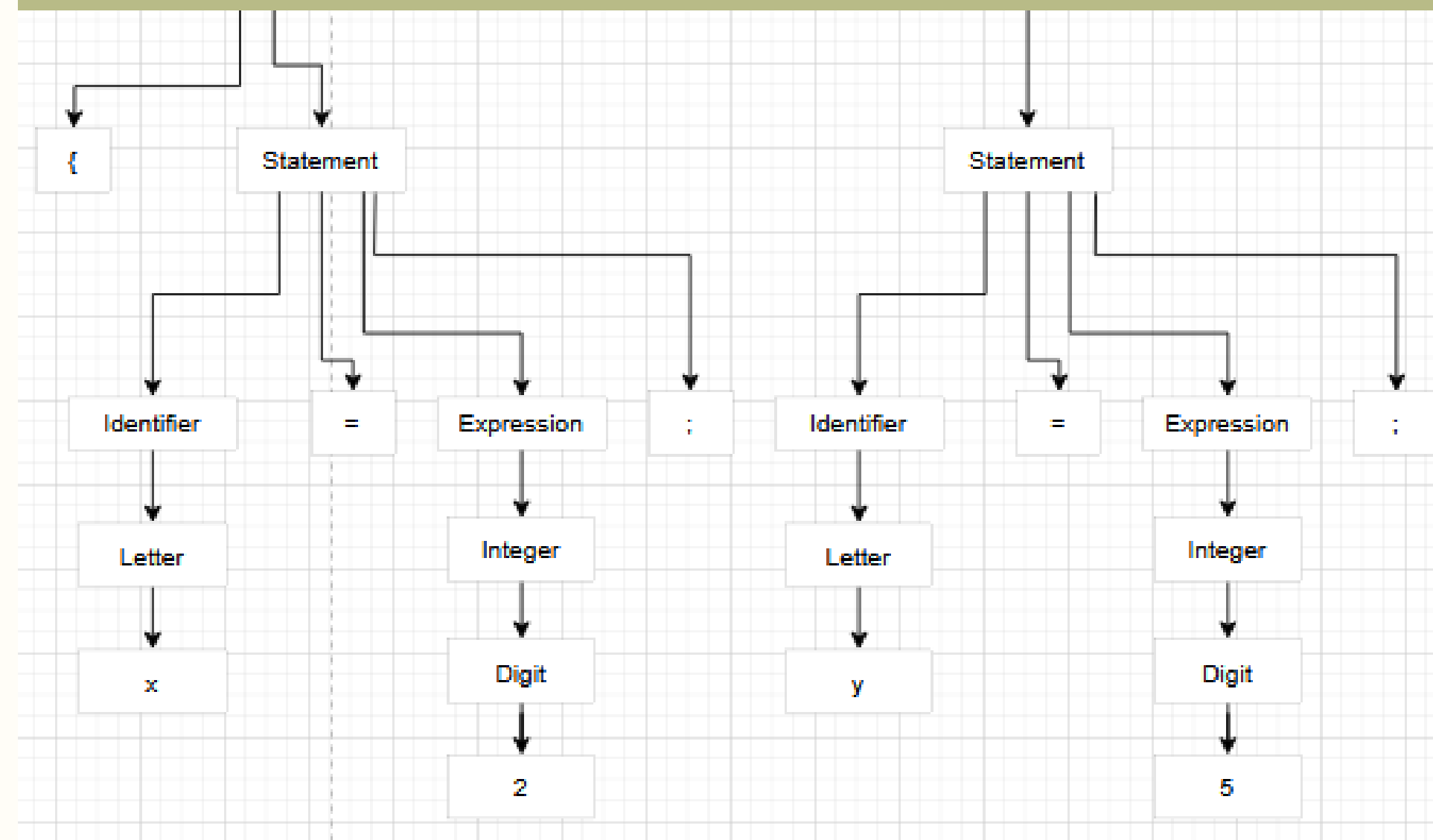
```
{
```

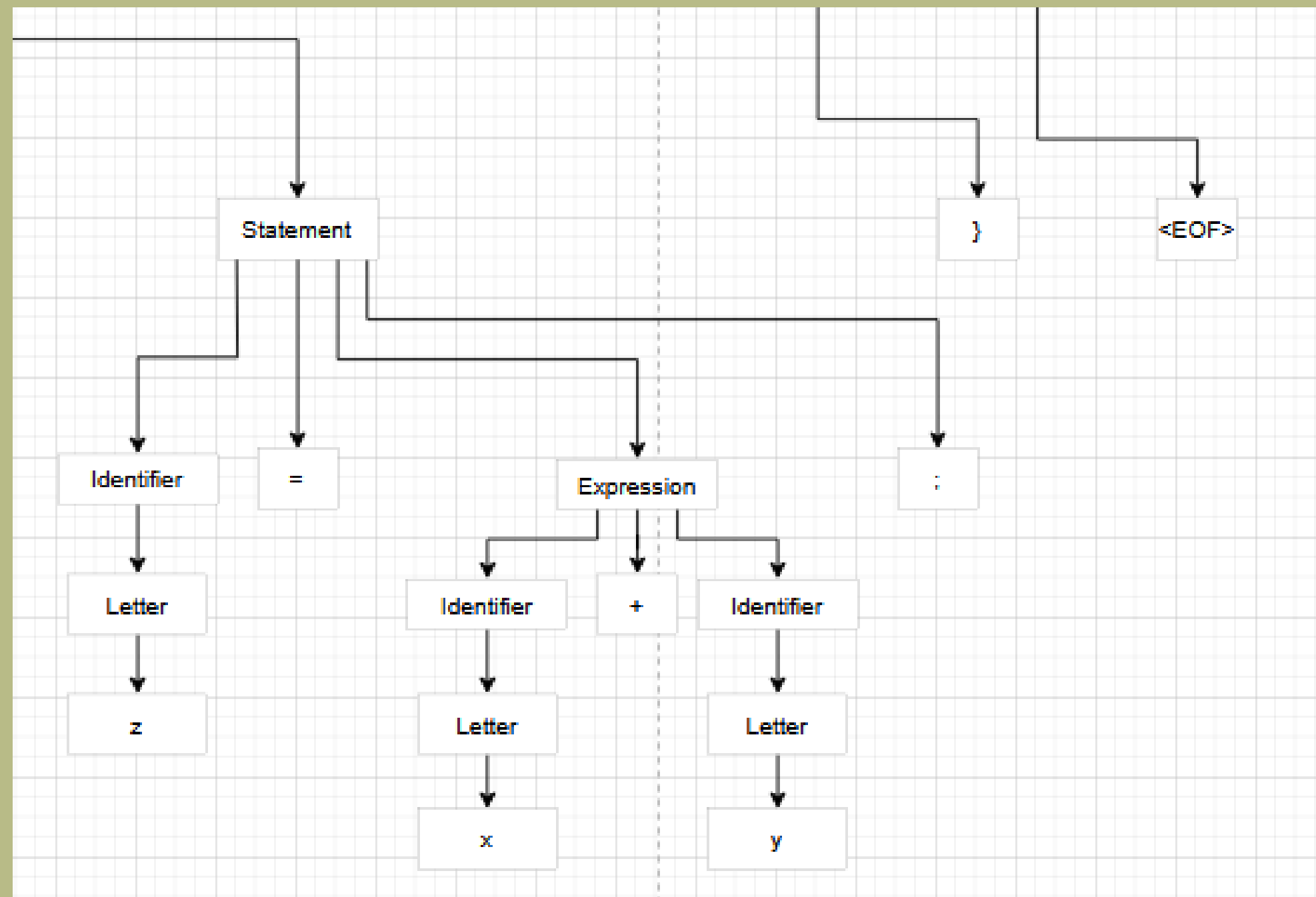
```
  x = 2;
```

```
  y = 5;
```

```
  z = x + y;
```

```
}
```





```
class prueba
```

```
int x;
```

```
int y;
```

```
int z;
```

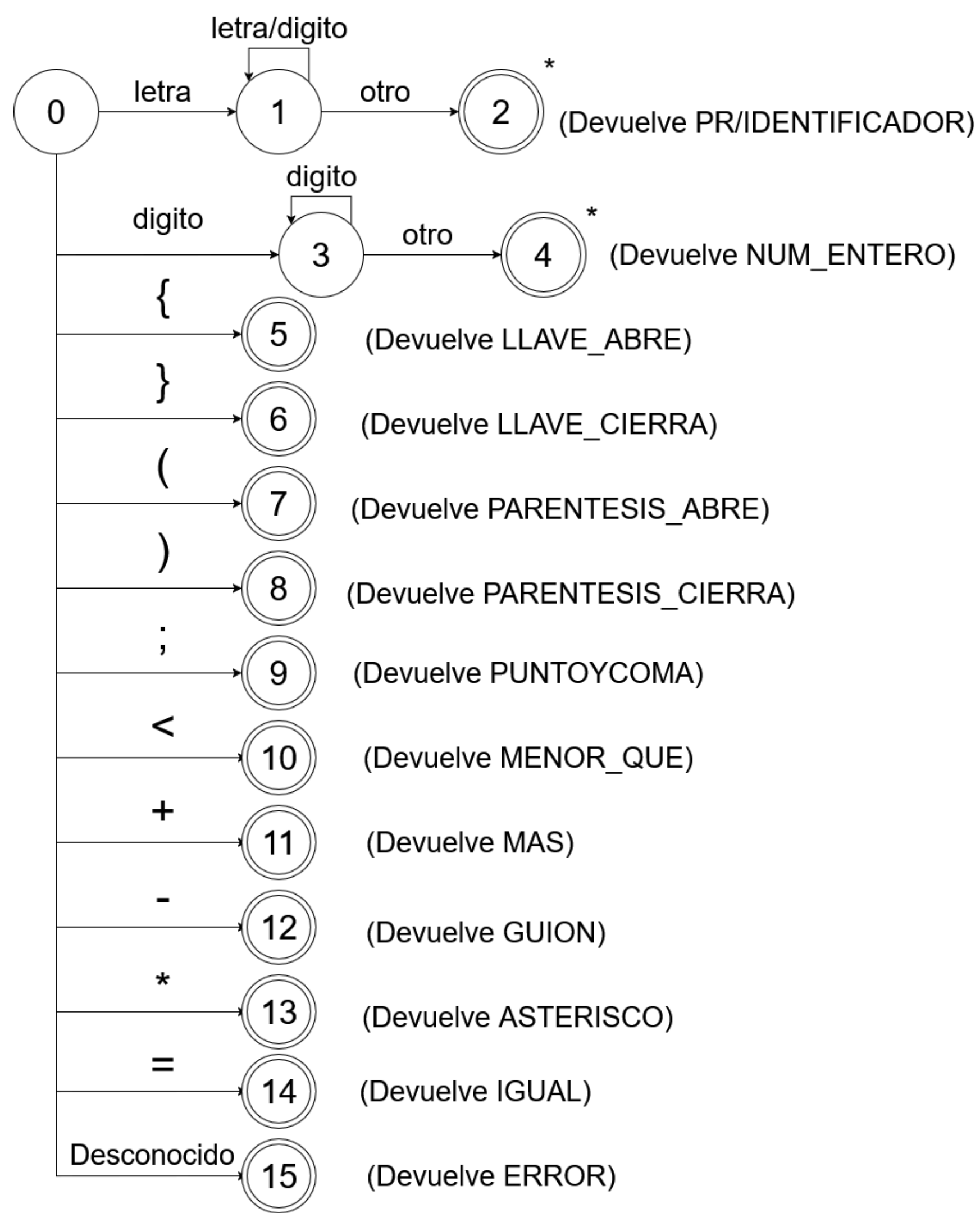
```
{
```

```
x = 2;
```

```
y = 5;
```

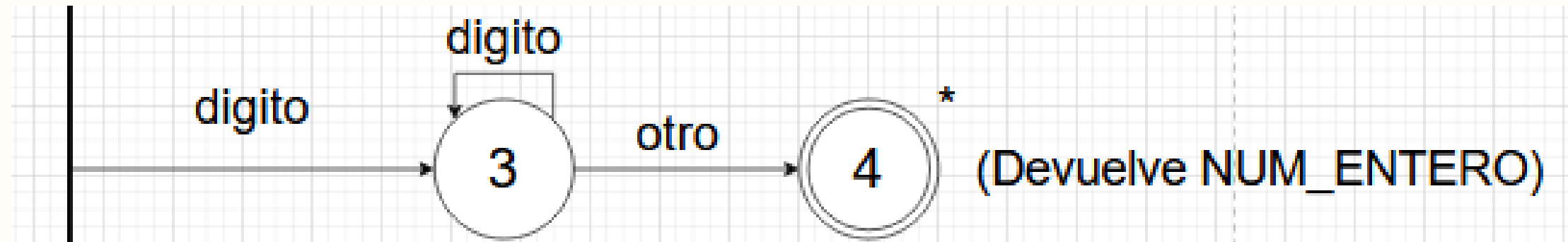
```
z = x + y;
```

```
}
```



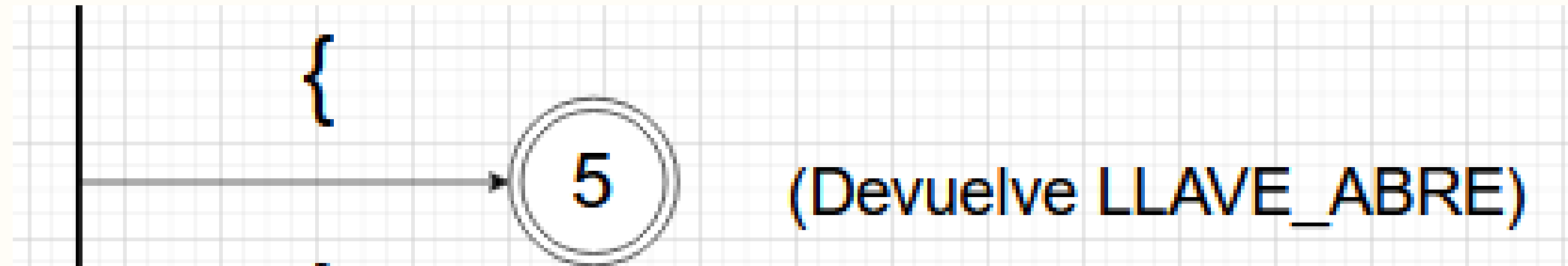
RECONOCEDORES DE TOKENS

CODIGO

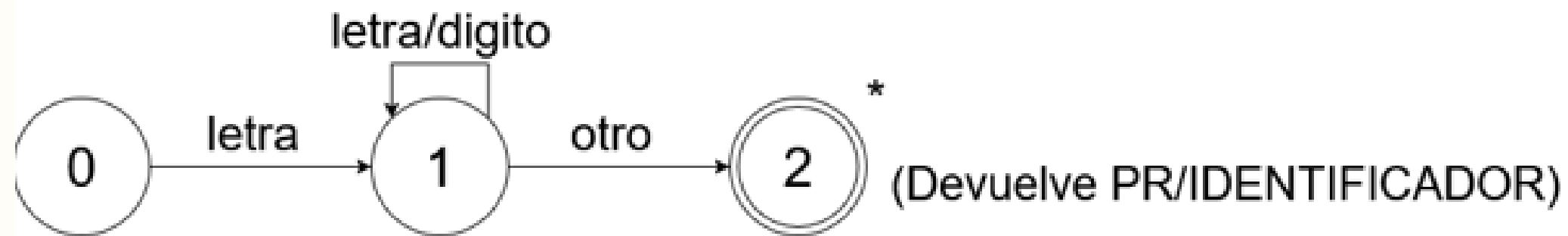


```
if (!Character.isDigit(verCaracter())) {  
    return null;  
}  
StringBuilder numero = new StringBuilder();  
while (Character.isDigit(verCaracter())) {  
    numero.append(verCaracter());  
    advance();  
}  
lexemaActual = numero.toString();  
return tipoID.NUM_ENTERO;
```

CODIGO



```
switch (verCaracter()) {  
    case '{':  
        advance();  
        lexemaActual = "{";  
        return tipoID.LLAVE_ABRE;
```



```

StringBuilder palabra = new StringBuilder();
if (!Character.isLetter(verCaracter())) {
    return null; }
while (Character.isLetterOrDigit(verCaracter())) {
    palabra.append(verCaracter());
    advance(); }
String str = palabra.toString();
lexemaActual = str;
switch (str) { ...
// Si no es ninguna palabra reservada cae por defecto a un identificador
default:
return tipoID.IDENTIFICADOR;
}
  
```

LISTA DE ID DE TOKENS PARA MÉTODOS DEL PARSER

```
// ----- Tokens -----  
  
EOF = 0  
  
class = 1  
  
boolean = 2  
  
int = 3  
  
while = 4  
  
true = 5  
  
false = 6  
  
println = 7  
  
IDENTIFICADOR = 8  
  
NUM_ENTERO = 9  
  
LLAVE_ABRE = 10  
  
LLAVE_CIERRA = 11  
  
PARENTESIS_ABRE = 12  
  
PARENTESIS_CIERRA = 13  
  
MENOR_QUE = 14  
  
MAS = 15  
  
GUION = 16  
  
ASTERISCO = 17  
  
PUNTOYCOMA = 18  
  
IGUAL = 19  
  
// -----
```

PARSER EJEMPLO 1 ASIGNACIÓN

```
private void parseStatement() throws IOException {  
    switch (tokenNum) {
```

```
case 8:
```

```
    // Asignacion "="
```

```
    eat(8);
```

```
    if (tokenNum != 19) { // =
```

```
        throw new RuntimeException("Se esperaba '=' después de identificador, se  
encontró "+ scanner.getLexema());
```

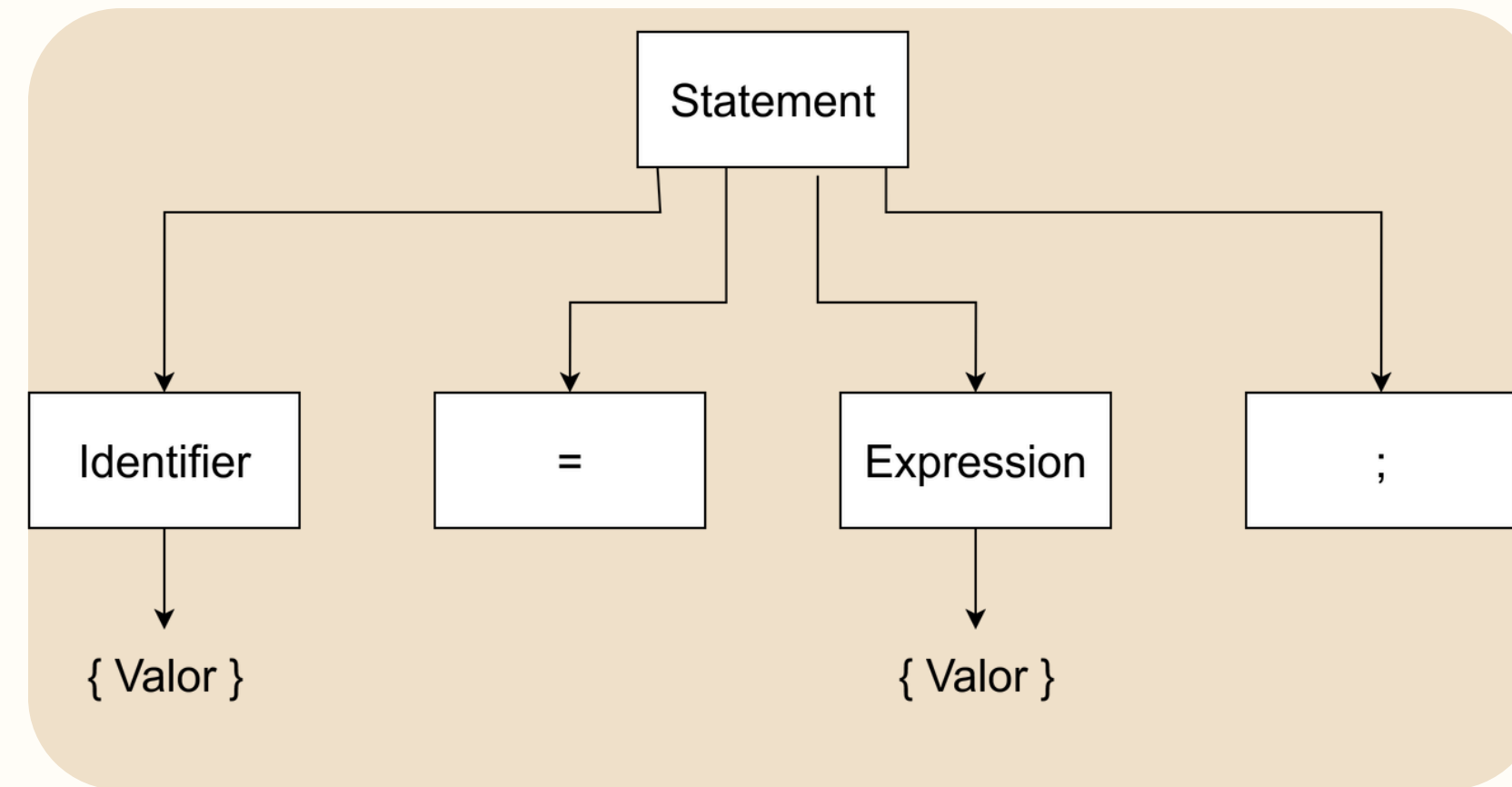
```
    }
```

```
    eat(19); // =
```

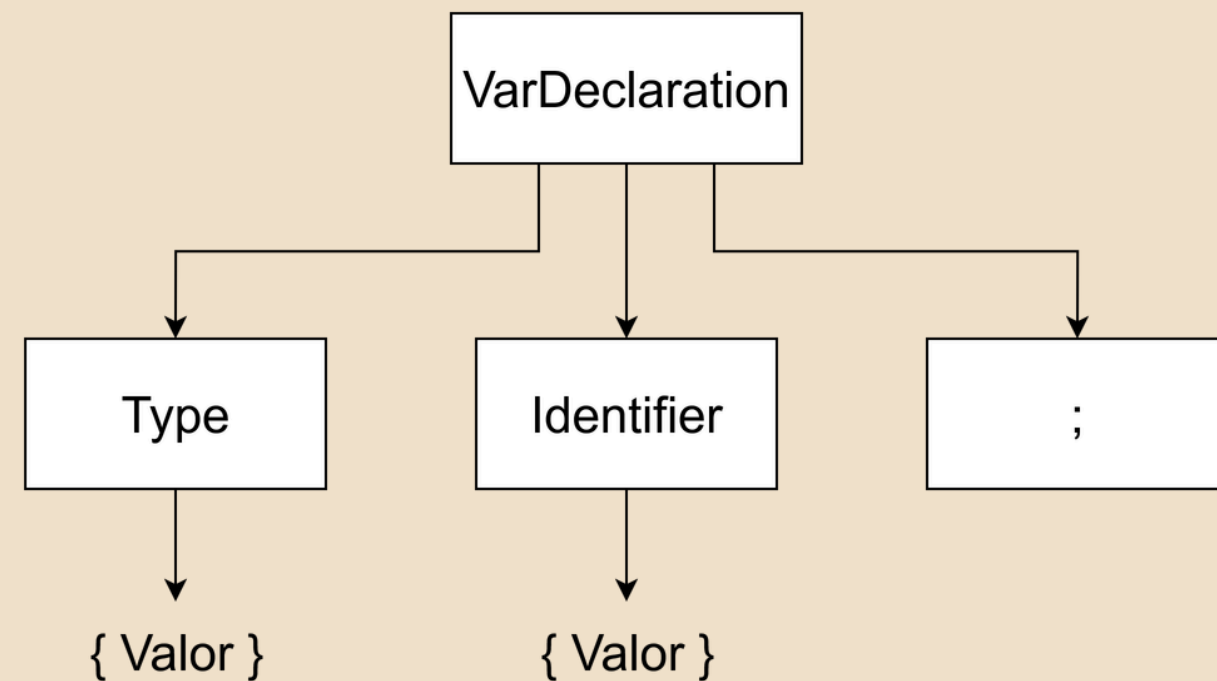
```
    parseExpression();
```

```
    eat(18); // ;
```

```
    break;
```



PARSER EJEMPLO 2 DECLARACIONES



```
// VarDeclaration ::= Type Identifier ;  
private void parseVarDeclaration() throws IOException {  
    parseType();  
    eat(8); // Identificador  
    eat(18); // ;  
}
```

```
private void parseType() throws IOException {  
    if (tokenNum == 2 || tokenNum == 3) { //O INT O BOOLEAN  
        eat(tokenNum);  
    } else {  
        throw new RuntimeException("Se esperaba un tipo  
(boolean/int) en línea "  
            + scanner.getNumeroDeLinea());  
    }  
}
```

ANALIZADOR SEMANTICO



INT X;
BOOLEAN X;

VERIFICAR QUE
NO SE DECLARE
LA MISMA
VARIABLE



CLASS
PRUEBA {
X = 3;
}

VERIFICAR QUE LAS
VARIABLES QUE ESTEN
DECLARADOS ANTES
DE SER USADAS



CLASS PRUEBA
INT X;
BOOLEAN Y;
{
X = X + Y
}

VERIFICAR QUE NO
HAYA DISTINTOS
TIPOS PARA UNA
OPERACIÓN

```

private void declaracionVariable() throws IOException {
    int tipoToken = tokenNum;
    eat(tipoToken);
    if (tokenNum == 8) { // IDENTIFICADOR
        String nombreVar = scanner.getLexema();
        String tipoStr;
        switch (tipoToken) {
            case 2:
                tipoStr = "boolean";
                break;
            case 3:
                tipoStr = "int";
                break;
            default:
                error("Tipo desconocido");
                tipoStr = "desconocido";
                break;
        } ...
    }
}

```



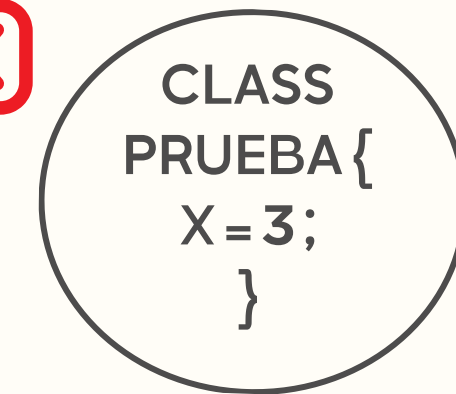
```

if (tablaSimbolos.containsKey(nombreVar))
{
    error("Variable '" + nombreVar + "' ya
    declarada");
}

```

case 8: // asignación

```
String nombreVar = scanner.getLexema();  
if (!tablaSimbolos.containsKey(nombreVar)) {  
    error("Variable '" + nombreVar + "' no declarada"); }  
String tipoVar = tablaSimbolos.get(nombreVar).tipo;  
eat(8); // identificador  
eat(19); // =  
ValorTipo valorExp = leerValorTipo();  
if (!tipoVar.equals(valorExp.tipo)) {  
    error("Error de tipo: se esperaba " + tipoVar + " pero  
se encontró " + valorExp.tipo);  
}  
if (!valorExp.esExpresion) {  
    tablaSimbolos.get(nombreVar).valor = valorExp.valor;  
}  
eat(18); // ;  
break;
```



VERIFICAR QUE LAS
VARIABLES QUE ESTEN
DECLARADOS ANTES
DE SER USADAS

```
while (tokenNum >= 14 && tokenNum <= 17) { // SI ES UN  
OPERADOR
```

```
    int op = tokenNum;
```

```
    eat(op);
```

```
    ValorTipo expresionDerecha = leerValorTipo();
```

```
    if (!tipo.equals("int") || !
```

```
expresionDerecha.tipo.equals("int")) {
```

```
        error("Type mismatch: ambos operandos de la  
expresión deben ser de tipo int");
```

```
    }
```

```
    tipo = "int";
```

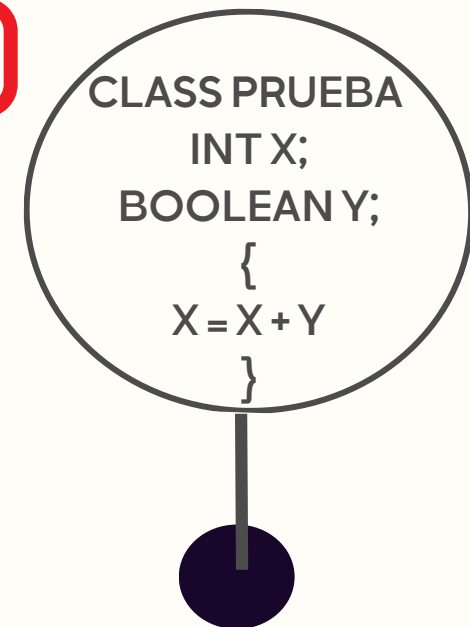
```
    valor = "";
```

```
    esExpresion = true;
```

```
}
```

```
return new ValorTipo(valor, tipo, esExpresion);
```

```
}
```



VERIFICAR QUE NO
HAYA DISTINTOS
TIPOS PARA UNA
OPERACIÓN

Código intermedio

CODIGO DE DOS DIRECCIONES

```
public class Triple {  
    public String operador;  
    public String arg1;  
    public String arg2;  
  
    public Triple(String operador, String arg1, String arg2) {  
        this.operador = operador;  
        this.arg1 = arg1;  
        this.arg2 = arg2;  
    }  
}
```

CODIGO EJEMPLO

```
class prueba
int x;
int y;
int z;
{
x = 2;
y = 5;
z = x + y;
}
```

SECUENCIA DE TRIPLES PRODUCIDAS

1. (=, x, 2)
2. (=, y, 5)
3. (+, x, y)
4. (=, z, (3))

```
for (int i = 0; i < triples.size(); i++) {
    Triple t = triples.get(i);
    switch (t.operador) {
        case "while":
            String etiqueta = "WHILE_" + etiquetaWhile;
            codigoFinal.append(String.format("%s_START:\n", etiqueta));
            // condicion
            codigoFinal.append("\tMOV AX, " + t.arg1 + "\n");
            codigoFinal.append("\tCMP AX, " + t.arg2 + "\n");
            codigoFinal.append(String.format("\tJGE %s_END\n\n", etiqueta));
            i++;
            while (i < triples.size() && !triples.get(i).operador.equals("while")) {
                Triple tCuerpo = triples.get(i);
                switch (tCuerpo.operador) {
```

```

        case "=":
            generarAsignacion(tCuerpo);
            break;
        case "+":
            generarOperacion(tCuerpo);
            break;
        case "println":
            codigoFinal.append("\t; println(" + tCuerpo.arg1 + ")\n");
            if (esVariableBooleana(tCuerpo.arg1)) {
                codigoFinal.append("\tMOV AL, " + tCuerpo.arg1 + "\n");
                codigoFinal.append("\tMOV AH, 0\n");
                codigoFinal.append("\tCALL PrintNumber\n");
            } else {
                codigoFinal.append("\tMOV AX, " + tCuerpo.arg1 + "\n");
                codigoFinal.append("\tCALL PrintNumber\n");
            }
            // retorno de carro 13
            codigoFinal.append("\tMOV DL, 13\n");
            codigoFinal.append("\tMOV AH, 02h\n");
            codigoFinal.append("\tINT 21h\n");
            // salto de línea 10
            codigoFinal.append("\tMOV DL, 10\n");
            codigoFinal.append("\tMOV AH, 02h\n");
            codigoFinal.append("\tINT 21h\n");
            codigoFinal.append("\n");
            break;
    }
    i++;
}
i--; // Retroceder uno porque el for lo avanza

// Volver al inicio del while
codigoFinal.append(String.format("\tJMP %s_START\n", etiqueta));
codigoFinal.append(String.format("%s_END:\n\n", etiqueta));
etiquetaWhile++;
break;

```



Muchas
GRACIAS

